

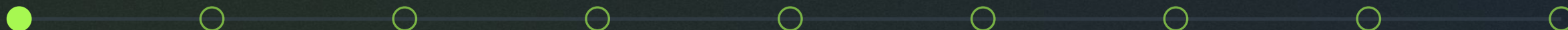


Lógica de Programação em JavaScript

Objetos, Dicionários e Maps

Instrutor Vinicius Dias

BE-JS-001 • Aula 3



Aula 3 | Objetos, Dicionários e Maps



Objetivos da aula de hoje

Ao final, você deve conseguir:

- Resolver o problema: modelar um contato como objeto e criar uma agenda de contatos com array de objetos.
- Entender e aplicar: Objetos JS (propriedades, acesso, mutação); Arrays associativos; Map e seus métodos.
- Implementar um dicionário de sinônimos usando Map e resolver exercícios de fixação sobre objetos e Maps.



Problema gerador

Agenda de Contatos & Dicionário de Sinônimos

Pergunta guia

Como modelar dados do mundo real (contatos, sinônimos) usando objetos e estruturas associativas em JavaScript?

Como vamos

- Estudo de caso 1: Agenda de contatos – array de objetos
- Estudo de caso 2: Dicionário de sinônimos – Map em JS
- Expositiva: Objetos JS, arrays associativos e Map
- Prático: Exercícios de fixação (objetos, Map)



Conceitos

Objetos e Dicionários

Hoje:

- Objetos JS (chave: valor)
- Arrays associativos e Map
- Agenda de contatos (array de objetos)
- Dicionário de sinônimos (Map)



Objetos em JavaScript

Objetos:

Objetos são coleções de pares chave-valor. A chave é sempre uma string (ou Symbol), o valor pode ser qualquer tipo. Acesse via ponto (obj.chave) ou colchetes (obj[“chave”]) – use colchetes quando a chave é dinâmica.

```
const contato = {  
  nome: "Ana Silva",  
  telefone: "991231234",  
};  
  
// Acessando propriedades  
console.log(contato.nome);      // "Ana Silva"  
console.log(contato["telefone"]); // "991231234"  
// Adicionando propriedade  
contato.email = "ana@email.com";  
// Deletando propriedade  
delete contato.nome;
```



Map em JavaScript

Map vs Objeto: use Map quando as chaves são dinâmicas, quando precisa manter ordem, ou quando vai iterar muitas vezes.

Map é uma estrutura chave-valor onde a chave pode ser qualquer tipo. Diferente de objetos comuns, mantém a ordem de inserção e tem `.size` nativo.

- `const sinonimos = new Map();`
- `sinonimos.set("feliz", ["alegre", "contente", "jovial"]);`
- `sinonimos.set("triste", ["melancólico", "abatido"]);`

// Acessando valores

`sinonimos.get("feliz");` // ["alegre", "contente"]

`sinonimos.has("triste");` // true

`sinonimos.size;` // 2 (quantidade de entradas)

// Removendo

`sinonimos.delete("triste");`

`sinonimos.clear();` // limpa tudo



Estudo de Caso 1: Agenda de Contatos

Shorthand { nome, tel, email } equivale a { nome: nome, tel: tel, email: email }. find() retorna o primeiro elemento que satisfaz a condição. Optional chaining (?.) evita erro se find() retornar undefined.

Modelamos cada contato como um objeto com propriedades nome, telefone e email.

Armazenamos todos os contatos em um array de objetos (a agenda).

```
const addressBook = [];  
  
function addContact(name, phone, email) {  
    const contact = { name, phone, email };  
    addressBook.push(contact);  
    return contact;  
}  
  
addContact("Ana", "991231234", "ana@email.com");  
addContact("Bruno", "991112222", "b@email.com");  
// Buscar contato por nome  
  
const found= agenda.find(c => c.nome === "Ana");  
console.log(found?.tel); // "991231234"
```



Estudo de Caso 2: Dicionário de Sinônimos

`join()` é método de array que une elementos com separador. O `for...of` em Map itera em pares `[chave, valor]` na ordem de inserção – garantia que objetos não oferecem.

- Usando Map para criar um dicionário de sinônimos interativo:

```
const synonyms = new Map();

function addSynonym(word, synonym) {
  if (!synonyms.has(word)) synonyms.set(word, []);
  synonyms.get(word).push(synonym);
}

function getSynonyms(word) {
  return synonyms.get(word) || [];
}

// Example
addSynonym("happy", "glad");
addSynonym("happy", "joyful");
addSynonym("fast", "quick");

console.log(getSynonyms("happy"));
```



Métodos de Objetos e Iteração

JavaScript fornece métodos nativos para inspecionar e iterar objetos:

- `Object.keys(obj)` – retorna array com as chaves
- `Object.values(obj)` – retorna array com os valores
 - a. `Object.entries(obj)` – retorna pares [chave, valor]
 - b. Spread: `{ ...obj }` – copia/mescla objetos
 - c. Exemplo com agenda:

```
const contato = { nome: "Ana", tel: "912 ... " };
```

```
Object.keys(contato); // ["nome", "tel"]
```

```
Object.values(contato); // ["Ana", "912 ... "]
```

```
Object.entries(contato);
```

- `// [["nome", "Ana"], ["tel", "912"]]`
- `const copia = { ...contato, email: "a@a.com" };`

Exercícios de fixação

Exercícios práticos: Objetos e Map

- grupo 1: Crie um objeto Produto com nome, preço e estoque. Adicione um método `aplicarDesconto(p)` que reduz o preço e funções para adicionar produto, buscar por nome e listar todos os produtos.
- grupo 2: Crie um dicionário de capitais usando Map: país → capital. Implemente busca (função que ache a capital pelo nome do país), função que lista as capitais em ordem alfabética e listagem (retorne todos os dados).
- Grupo 3: Crie um objeto usuário com nome, idade, cidade e habilidades (array). Imprima “Olá, NOME! Você mora em CIDADE.” Depois adicione uma habilidade nova, atualize a cidade e mostre no console se é “Menor de idade” ou “Maior de idade”. Crie também a função `resumoUsuario(usuario)` que retorna uma string com um resumo do perfil.

Feito é melhor que perfeito: entregue uma versão simples, depois melhore!



Exemplo (código)

Versão completa para discutir e evoluir

```
const products = [];  
  
function addProduct(product) {  
  products.push(product);  
}  
  
function findProductByName(name) {  
  return products.find((p) => p.name === name) || null;  
}  
  
function setDiscount(productName, discount) {  
  const product = findProductByName(productName)  
  product.price = product.price * discount  
}  
  
function listProducts() {  
  return products;  
}  
  
// Example  
addProduct({ name: "T-Shirt", price: 50, stock: 10 });  
addProduct({ name: "Sneakers", price: 200, stock: 5 });
```



Exemplo (código)

Versão completa para discutir e evoluir

```
const capitals = new Map();

function setCapital(country, capital) {
  capitals.set(country, capital);
}

function getCapital(country) {
  return capitals.get(country) || null;
}

function listCapitals() {
  const result = [];
  for (const [country, capital] of capitals.entries()) {
    result.push({ country, capital });
  }
  return result;
}

// Example
setCapital("Brazil", "Brasília");
setCapital("Portugal", "Lisbon");
setCapital("France", "Paris");

console.log(getCapital("Portugal")); // Lisbon
console.log(getCapital("Italy")); // null
console.log(listCapitals());
```



Pesquisa Individual

Pesquisa e Referências

Leituras recomendadas:

- MDN Object:
developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/Object
- Stack Overflow, fóruns, YouTube

Pergunta de pesquisa (5–10 min)

Qual a diferença entre Map e um objeto comum ({}) em JavaScript? Quando usar cada um? Traga exemplos.



Próxima aula

Aula 4

Funções avançadas: escopo, closure, callbacks e arrow functions.

Para chegar bem:

- Traga dúvidas sobre objetos, Map e os exercícios
- Complete os 3 exercícios de fixação
- Revise o slide de Conceitos e marque o que ficou confuso



Obrigado